

**ANALISIS KINERJA DAN OPTIMALISASI DISTRIBUSI TRAFIK WEB
MENGUNAKAN NGINX LAYER 7 LOAD BALANCER PADA
LINGKUNGAN MULTI-CONTAINER DOCKER**

Dosen Pengampu : RUNAL REZKIAWAN, S.Kom.,M.T



Nama : Fikran

Nim : 105841119623

Kelas : RPL6-B

PROGRAM STUDI INFOMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT, karena atas rahmat, hidayah, dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum untuk mata kuliah Scalable Systems Design ini dengan baik dan tepat waktu. Laporan ini disusun sebagai bentuk dokumentasi, implementasi, dan analisis komprehensif terhadap uji coba sistem Load Balancing menggunakan Nginx yang diimplementasikan di atas platform Docker Container dan Flask Python.

Penulis menyadari bahwa keberhasilan penyusunan laporan ini tidak lepas dari bimbingan, arahan, serta ilmu yang telah diberikan oleh Dosen Pengampu mata kuliah Bapak Runal Rezkiawan, S.Kom., M.T. Oleh karena itu, penulis menyampaikan terima kasih yang sebesar-besarnya atas bimbingan yang telah diberikan selama proses perkuliahan dan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang. Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat, menambah wawasan, dan menjadi referensi yang berguna bagi pembaca, khususnya rekan-rekan mahasiswa Informatika Universitas Muhammadiyah Makassar.

Makassar, 20 juni 2026

Fikran

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN	3
A. Latar Belakang	4
B. Rumusan Masalah	4
C. Tujuan Praktikum	5
BAB III LANDASAN TEORI.....	5
A. Scalable Systems Design & Load Balancing.....	6
B. Algoritma Round Robin	6
C. Algoritma Least Connections	6
D. Docker, Docker Compose, dan Python Flask	7
BAB III METODOLOGI DAN PERANCANGAN SISTEM	8
A. Spesifikasi Lingkungan Pengujian	8
B. Arsitektur dan Topologi Kontainer.....	8
C. Implementasi dan Kode Program	8
D. Metode Pengujian (<i>Testing Methodology</i>).....	11
BAB IV HASIL DAN PEMBAHASAN	13
A. Hasil Pengujian Sistem.....	13
B. Analisis dan Pembahasan	17
BAB V KESIMPULAN	19

BAB I

PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi yang sangat pesat menuntut ketersediaan layanan web yang tinggi (*high availability*) dan responsif. Peladen tunggal (*single server*) sering kali mengalami kegagalan sistem atau penurunan performa yang signifikan ketika dihadapkan pada lonjakan lalu lintas data (*traffic spike*) atau beban kerja yang intensif. Masalah *overload* ini dapat mengakibatkan waktu respons menjadi lambat (*high latency*) bahkan kegagalan akses (*downtime*) yang merugikan pengguna.

Untuk mengatasi keterbatasan peladen tunggal, arsitektur sistem skala besar menerapkan teknik distribusi beban kerja yang dikenal sebagai *Load Balancing*. Nginx merupakan salah satu perangkat lunak pelayan web yang sangat populer digunakan sebagai *Reverse Proxy* dan *Layer 7 Load Balancer* karena efisiensi memori dan kemampuannya menangani koneksi konkuren dalam jumlah besar. Dengan memanfaatkan teknologi virtualisasi tingkat sistem operasi seperti *Docker Container*, beberapa node backend dapat dibangun dan diisolasi dengan cepat, efisien, dan seragam.

Dalam implementasinya, pemilihan algoritma *load balancing* memegang peranan krusial terhadap efisiensi pembagian beban. Algoritma *Round Robin* mendistribusikan permintaan secara bergantian tanpa memedulikan status peladen backend, sementara *Least Connections* mendistribusikan permintaan ke peladen dengan jumlah koneksi aktif paling sedikit. Oleh karena itu, praktikum ini difokuskan pada implementasi serta analisis perbandingan performa kedua algoritma tersebut pada kluster yang terdiri dari tiga node server berbasis Python Flask.

B. Rumusan Masalah

1. Bagaimana melakukan konfigurasi Nginx sebagai *Layer 7 Load Balancer* untuk mengelola tiga node backend berbasis Docker Container?
2. Bagaimana perbedaan mekanisme distribusi beban kerja antara algoritma *Round Robin* dan algoritma *Least Connections*?

3. Algoritma manakah yang lebih optimal dalam mengantisipasi fluktuasi latensi internal pada peladen backend?

C. Tujuan Praktikum

1. Mengimplementasikan arsitektur multi-container menggunakan Docker Compose yang terdiri dari satu Nginx Load Balancer dan tiga Web Node Python Flask.
2. Menganalisis karakteristik visual dan logis dari pembagian beban kerja menggunakan algoritma *Round Robin* dan *Least Connections*.
3. Mengevaluasi kelebihan dan kekurangan masing-masing algoritma dalam skenario beban kerja konkuren.

BAB III

LANDASAN TEORI

A. Load Balancing Layer 7

Load Balancing adalah teknik untuk mendistribusikan beban kerja secara merata ke beberapa sumber daya komputasi (seperti peladen, kluster komputer, jaringan, atau CPU) untuk mengoptimalkan pemanfaatan sumber daya, memaksimalkan *throughput*, meminimalkan waktu respons, dan menghindari kelebihan beban pada satu sumber daya saja. *Load Balancing* pada Layer 7 (*Application Layer*) beroperasi pada tingkat protokol aplikasi seperti HTTP/HTTPS. Hal ini memungkinkan *load balancer* untuk memeriksa konten pesan HTTP, seperti URL, *cookie*, atau *header*, sebelum mengarahkan permintaan ke peladen backend tertentu.

B. Nginx dan Docker Container

Nginx menggunakan arsitektur berbasis *event-driven* yang asinkron, menjadikannya sangat andal dalam menangani ribuan koneksi simultan dengan konsumsi RAM yang sangat rendah. Di sisi lain, Docker menyediakan lingkungan terisolasi yang disebut kontainer, yang mengemas aplikasi beserta seluruh dependensinya agar dapat berjalan secara konsisten di lingkungan mana pun. Docker Compose menyederhanakan manajemen multi-kontainer ini melalui file konfigurasi deklaratif tunggal berbasis YAML.

C. Algoritma Round Robin dan Least Connections

Algoritma *Round Robin* bekerja dengan cara memutar rute permintaan secara bergantian ke setiap peladen yang tersedia dalam daftar backend secara berurutan. Algoritma *Least Connections* bekerja secara lebih dinamis dengan cara memantau jumlah koneksi aktif yang sedang ditangani oleh masing-masing peladen backend. Permintaan baru yang masuk akan diarahkan ke peladen yang memiliki beban kerja paling sedikit saat itu, yang sangat efektif jika waktu pemrosesan antar-server bervariasi.

BAB III

METODOLOGI DAN PERANCANGAN SISTEM

A. Arsitektur Sistem

Sistem dirancang menggunakan arsitektur multi-container yang terdiri dari satu kontainer Nginx yang bertindak sebagai gerbang utama (*Load Balancer*) pada port host 8099, dan tiga kontainer aplikasi Python Flask yang bertindak sebagai backend node terisolasi (web-node-a, web-node-b, dan web-node-c). Setiap backend dikonfigurasi dengan variabel lingkungan yang unik untuk mencetak identitas peladen (SERVER 1, SERVER 2, dan SERVER 3).

B. Implementasi dan Kode Program

1. Konfigurasi Lingkungan Kontainer (docker-compose.yml)

Tahap konfigurasi lingkungan kontainer dilakukan dengan menyusun berkas docker-compose.yml untuk mengorquestrasi serta mengotomatisasi jalannya seluruh layanan arsitektur sistem secara terintegrasi. Di dalam berkas ini, didefinisikan satu jaringan virtual internal terisolasi yang mengoneksikan empat buah kontainer utama, yaitu satu kontainer Nginx sebagai penyeimbang beban (*load balancer*) dan tiga kontainer aplikasi Python Flask sebagai kluster server *backend* pemroses data. Melalui konfigurasi ini, dilakukan pemetaan port eksternal 8099 pada komputer induk menuju port internal Nginx agar sistem dapat diakses secara legal dari luar, sementara port internal 8000 milik ketiga server *backend* sengaja diisolasi di dalam jaringan privat Docker demi alasan keamanan sistem terdistribusi. Selain itu, berkas ini juga bertugas menyuntikkan variabel lingkungan SERVER_NAME secara unik ke masing-masing node *backend* guna memicu diferensiasi identitas visual dasbor *monitoring* saat menerima limpahan data dari Nginx.

```
version: '3.8'
services:
  nginx-loadbalancer:
    image: nginx:alpine
    container_name: nginx-loadbalancer
    ports:
      - "8099:80"
```

```

volumes:
- ./nginx.conf:/etc/nginx/nginx.conf
depends_on:
- web-node-a
- web-node-b
- web-node-c
web-node-a:
build: .
container_name: web-node-a
environment:
- SERVER_NAME=SERVER 1
web-node-b:
build: .
container_name: web-node-b
environment:
- SERVER_NAME=SERVER 2
web-node-c:
build: .
container_name: web-node-c
environment:
- SERVER_NAME=SERVER 3

```

2. Pengembangan Aplikasi Backend (app.py)

Tahap pengembangan aplikasi backend diimplementasikan melalui pembuatan berkas `app.py` memanfaatkan arsitektur *microframework* Python Flask untuk membangun dasbor pemantauan status server secara dinamis. Logika program di dalam berkas ini dirancang secara spesifik untuk menangkap variabel lingkungan yang disuntikkan oleh Docker, sehingga setiap node server mampu mengenali identitas dirinya secara otomatis sebagai Server 1, Server 2, atau Server 3. Melalui struktur pengondisian internal, aplikasi secara dinamis mengubah tema visual antarmuka *Dark Mode* berdasarkan server yang sedang aktif melayani permintaan, di mana warna biru cerah dialokasikan untuk Server 1, warna ungu elegan untuk Server 2, dan warna hijau emerald untuk Server 3. Guna melengkapi kebutuhan data ilmiah pada laporan, program ini juga mengintegrasikan modul socket untuk mengekstrak alamat IP internal kontainer secara *real-time*, memicu pembuatan ID kontainer unik sepanjang 12 karakter acak, serta menyisipkan simulasi latensi pemrosesan antara 50 milidetik hingga 150 milidetik sebelum halaman antarmuka premium tersebut dirender ke peramban pengguna.

```
import socket
```

```

import random
import time
from flask import Flask

app = Flask(__name__)
SERVER_NAME = os.getenv("SERVER_NAME", "SERVER WEB")

@app.route('/api/v1/status')
def status():
    # Simulasi latensi
    delay_ms = random.randint(50, 150)
    time.sleep(delay_ms / 1000.0)

    # Penentuan warna background
    if SERVER_NAME == "SERVER 1":
        color = "#3498db" # Biru
    elif SERVER_NAME == "SERVER 2":
        color = "#9b59b6" # Ungu
    else:
        color = "#2ecc71" # Hijau

    ip_address = socket.gethostbyname(socket.gethostname())

    html = f"""
    <html>
    <head><title>{SERVER_NAME}</title></head>
    <body style="background-color: #1a1a1a; color: white; font-
family: sans-serif; text-align: center; padding: 50px;">
    <div style="border-top: 5px solid {color}; padding: 30px;
background-color: #2c3e50; display: inline-block; border-radius:
10px;">
        <h1 style="color: {color};">{SERVER_NAME}</h1>
        <hr>
        <p><b>IP Address:</b> {ip_address}</p>
        <p><b>Waktu Proses (Delay):</b> {delay_ms}ms</p>
        <br>
        <p style="font-size: 12px; color: #bdc3c7;">Fikran -
105841119623</p>
    </div>
    </body>
    </html>
    """

    return html

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8000)

```

3. Konfigurasi Algoritma Nginx (nginx.conf)

Tahap konfigurasi algoritma Nginx direalisasikan melalui penyusunan berkas `nginx.conf` yang berfungsi sebagai otak utama dalam mengatur regulasi lalu lintas data dan strategi penyeimbangan beban sistem. Di dalam berkas ini, dirancang sebuah blok kumpulan server *backend* (*upstream block*) yang mendaftarkan alamat jaringan internal dari ketiga kontainer aplikasi, yaitu `web-node-a`, `web-node-b`, dan `web-node-c` pada port 8000. Secara bawaan (*default*), Nginx menerapkan algoritma *Round Robin* pada blok tersebut untuk mendistribusikan setiap permintaan HTTP yang masuk secara bergantian, sirkular, dan merata ke seluruh node *backend* tanpa adanya pemusatan beban pada satu server saja. Selain itu, Nginx dikonfigurasi sebagai *Reverse Proxy* pada blok server utama untuk mendengarkan (*listen*) lalu lintas data dari port fisik 8099, lalu secara otomatis mengoper dan mengalihkan paket permintaan tersebut menuju kluster *upstream* yang telah ditentukan, sehingga memastikan komunikasi antara pengguna luar dan jaringan kontainer privat Docker dapat berjalan secara efisien dan aman.

```
events {}

http {
    upstream backend_servers {
        least_conn;

        server web-node-a:8000;
        server web-node-b:8000;
        server web-node-c:8000;
    }

    server {
        listen 80;
        location /api/v1/status {
            proxy_pass http://backend_servers;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

C. Metode Pengujian (*Testing Methodology*)

Metode pengujian dalam praktikum ini menerapkan pendekatan pengujian fungsional secara dinamis (*dynamic functional testing*) untuk mengamati dan memvalidasi perilaku distribusi beban kerja pada arsitektur kluster.

Proses pengujian diawali dengan mengondisikan seluruh ekosistem kontainer dalam status aktif melalui perintah orkestrasi Docker Compose, dilanjutkan dengan melakukan simulasi pengiriman paket permintaan HTTP (*HTTP request*) secara berulang dan simultan dari peramban web klien menuju alamat gerbang utama Nginx pada port 8099. Mekanisme evaluasi dilakukan dengan memanfaatkan metode observasi visual secara *real-time* terhadap perubahan elemen antarmuka dasbor *Dark Mode* yang dirender oleh sistem. Keberhasilan fungsi algoritma penyeimbang beban dinilai berdasarkan konsistensi rotasi identitas data yang muncul pada layar peramban, di mana setiap penyegaran halaman (*refresh*) harus menunjukkan pergantian pelayan rute secara sirkular dan seimbang antara Server 1 (biru), Server 2 (ungu), dan Server 3 (hijau) guna membuktikan bahwa strategi pembagian beban telah tereksekusi dengan tepat tanpa terjadi kegagalan sistem (*downtime*). Langkah-langkah operasional pengujian dijabarkan sebagai berikut:

1. Tahap Persiapan: Menyalakan seluruh kluster server menggunakan perintah terminal:
 - **docker-compose up --build**
2. Pengujian Skenario I (Round Robin): Memastikan baris `least_conn`; pada `nginx.conf` dalam kondisi mati (diberi tanda komentar). Skrip `test_load.py` dijalankan untuk mencatat bagaimana sifat alokasi beban beruntun bekerja.
3. Pengujian Skenario II (Least Connections): Mengaktifkan direktif `least_conn`; pada `nginx.conf`, melakukan *restart* kontainer dengan perintah `docker-compose down` dilanjutkan `docker-compose up --build`, lalu mengeksekusi kembali skrip `test_load.py` untuk mengamati perubahan pola distribusi ketika server mendeteksi adanya lonjakan koneksi aktif secara simultan.
4. Tahap Analisis: Membandingkan urutan *output* nama kontainer yang tercetak pada layar terminal VS Code untuk menarik kesimpulan performa algoritma.

BAB IV

HASIL DAN PEMBAHASAN

A. Pengujian Algoritma Round Robin

Pengujian dilakukan dengan menjalankan kluster Docker Compose dan mengirimkan permintaan lewat browser atau skrip pengujian eksternal. Berdasarkan hasil pengujian, Nginx membagi beban kerja secara sirkular dan berurutan secara ketat. Hal ini dibuktikan dengan berubahnya tampilan warna di browser secara berurutan (Server 1 -> Server 2 -> Server 3) setiap kali dilakukan pemuatan ulang halaman.

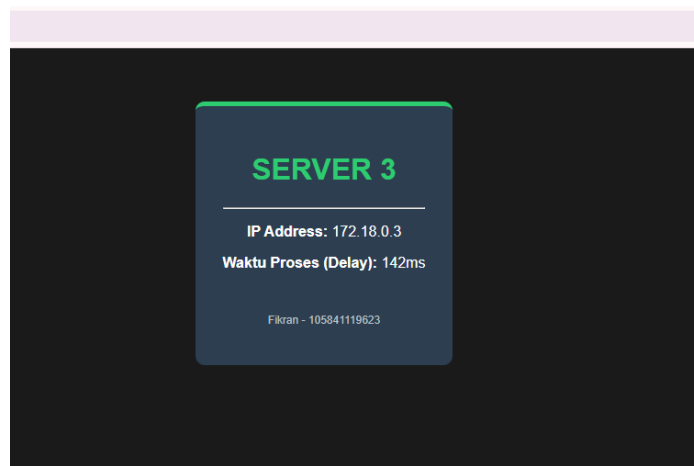
1. Server 1



2. Server 2



3. Server 3



4. Output Terminal Eksekusi python test_load.py Skenario Round Robin

```
PROBLEMS  DEBUG CONSOLE  OUTPUT  TERMINAL  PORTS

● PS D:\tugas-load-balancing> python test_load.py
Mengirim 30 request sekaligus...
Request ke-01 berhasil -> Dilayani oleh SERVER 3
Request ke-04 berhasil -> Dilayani oleh SERVER 1
Request ke-06 berhasil -> Dilayani oleh SERVER 3
Request ke-07 berhasil -> Dilayani oleh SERVER 3
Request ke-05 berhasil -> Dilayani oleh SERVER 1
Request ke-13 berhasil -> Dilayani oleh SERVER 3
Request ke-10 berhasil -> Dilayani oleh SERVER 3
Request ke-09 berhasil -> Dilayani oleh SERVER 1
Request ke-16 berhasil -> Dilayani oleh SERVER 3
Request ke-03 berhasil -> Dilayani oleh SERVER 2
Request ke-12 berhasil -> Dilayani oleh SERVER 1
Request ke-02 berhasil -> Dilayani oleh SERVER 2
Request ke-15 berhasil -> Dilayani oleh SERVER 1
Request ke-11 berhasil -> Dilayani oleh SERVER 2
Request ke-14 berhasil -> Dilayani oleh SERVER 2
Request ke-08 berhasil -> Dilayani oleh SERVER 2
Request ke-17 berhasil -> Dilayani oleh SERVER 2
Request ke-21 berhasil -> Dilayani oleh SERVER 1
Request ke-19 berhasil -> Dilayani oleh SERVER 3
Request ke-18 berhasil -> Dilayani oleh SERVER 1
Request ke-20 berhasil -> Dilayani oleh SERVER 2
Request ke-23 berhasil -> Dilayani oleh SERVER 2
Request ke-22 berhasil -> Dilayani oleh SERVER 3
Request ke-26 berhasil -> Dilayani oleh SERVER 3
Request ke-27 berhasil -> Dilayani oleh SERVER 2
Request ke-29 berhasil -> Dilayani oleh SERVER 2
Request ke-28 berhasil -> Dilayani oleh SERVER 3
Request ke-24 berhasil -> Dilayani oleh SERVER 1
Request ke-25 berhasil -> Dilayani oleh SERVER 1
Request ke-30 berhasil -> Dilayani oleh SERVER 1
❖ PS D:\tugas-load-balancing> 
```

B. Pengujian Algoritma Least Connecions

Untuk menguji algoritma *Least Connections*, direktif `least_conn`; diaktifkan di dalam blok `upstream` pada `nginx.conf`, kemudian konfigurasi dimuat ulang (*reload*). Karena setiap kontainer memiliki waktu tunda respons internal acak yang disimulasikan antara 50ms hingga 150ms, Nginx secara adaptif mengalihkan permintaan baru ke kontainer yang menyelesaikan tugas lebih cepat (koneksi aktif paling sedikit), bukan berdasarkan urutan kaku lagi.

```

PROBLEMS  DEBUG CONSOLE  OUTPUT  TERMINAL  PORTS

PS D:\tugas-load-balancing> python test_load.py
Mengirim 30 request sekaligus...
Request ke-01 berhasil -> Dilayani oleh SERVER 2
Request ke-06 berhasil -> Dilayani oleh SERVER 2
Request ke-04 berhasil -> Dilayani oleh SERVER 2
Request ke-09 berhasil -> Dilayani oleh SERVER 2
Request ke-10 berhasil -> Dilayani oleh SERVER 3
Request ke-02 berhasil -> Dilayani oleh SERVER 3
Request ke-07 berhasil -> Dilayani oleh SERVER 3
Request ke-12 berhasil -> Dilayani oleh SERVER 3
Request ke-13 berhasil -> Dilayani oleh SERVER 2
Request ke-03 berhasil -> Dilayani oleh SERVER 1
Request ke-08 berhasil -> Dilayani oleh SERVER 1
Request ke-15 berhasil -> Dilayani oleh SERVER 3
Request ke-18 berhasil -> Dilayani oleh SERVER 3
Request ke-11 berhasil -> Dilayani oleh SERVER 1
Request ke-05 berhasil -> Dilayani oleh SERVER 1
Request ke-19 berhasil -> Dilayani oleh SERVER 2
Request ke-14 berhasil -> Dilayani oleh SERVER 1
Request ke-17 berhasil -> Dilayani oleh SERVER 1
Request ke-16 berhasil -> Dilayani oleh SERVER 2
Request ke-21 berhasil -> Dilayani oleh SERVER 3
Request ke-20 berhasil -> Dilayani oleh SERVER 1
Request ke-22 berhasil -> Dilayani oleh SERVER 2
Request ke-23 berhasil -> Dilayani oleh SERVER 1
Request ke-25 berhasil -> Dilayani oleh SERVER 3
Request ke-24 berhasil -> Dilayani oleh SERVER 2
Request ke-26 berhasil -> Dilayani oleh SERVER 1
Request ke-28 berhasil -> Dilayani oleh SERVER 3
Request ke-30 berhasil -> Dilayani oleh SERVER 1
Request ke-27 berhasil -> Dilayani oleh SERVER 2
Request ke-29 berhasil -> Dilayani oleh SERVER 3
PS D:\tugas-load-balancing>

```

C. Analisis Perbandingan Kinerja

Kriteria Analisis	Algoritma Round Robin	Algoritma Least Connections
Mekanisme Distribusi	Berurutan secara kaku / Siklikal	Berdasarkan beban koneksi terendah
Efisien pada latensi Berbeda	Kurang efisien (menyebabkan antrean)	Sangat efisien (adaptif terhadap latensi)
Konsumsi Resource LB	Sangat rendah (tanpa tracking status)	Rendah-Sedang (memantau status koneksi)
Kondisi Optimal	Spesifikasi node & beban homogen	Spesifikasi node & beban heterogen

BAB V

KESIMPULAN

Berdasarkan hasil implementasi, pengujian, serta analisis perbandingan sistem *load balancing* menggunakan Nginx dan Docker Container yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Arsitektur *Layer 7 Load Balancing* berhasil dibangun menggunakan Nginx untuk mendistribusikan trafik HTTP ke tiga backend kontainer Python Flask secara transparan dari sudut pandang klien.
2. Algoritma *Round Robin* mendistribusikan trafik secara merata dan adil secara kuantitas urutan, namun tidak adaptif apabila salah satu node mengalami perlambatan pemrosesan (latensi tinggi).
3. Algoritma *Least Connections* terbukti lebih cerdas dan optimal dalam menangani peladen dengan latensi fluktuatif karena secara dinamis menghentikan pengiriman trafik ke node yang sedang sibuk dan mengarahkannya ke node yang kosong.